

Organising Java Classes

Objectives

- Packages
- Importing Classes
- Options for compiling and running classes



Organising Classes

- Java classes are organised into **packages**
- Packages are grouped according to functionality
- Packaging is done to avoid name clashes
- The Java library is divided up into predefined packages
- Your Java classes should all be in your own packages

Common Packages in Java

Package	Contains
<code>java.lang</code>	Language classes, including System, Math, String
<code>java.text</code>	Text formatting classes for dates, times, and currencies.
<code>java.sql</code>	Database access classes.
<code>java.util</code>	Utility classes for various generic functionality such as working with collections.

Import

- The **import** keyword is used to specify which classes you wish to use
- You can either import individual classes or whole packages

```
import java.text.*;           // imports all classes in java.text package
import java.io.PrintWriter;   // imports only one class from the package

public class Account {
    Formatter formatter;
    PrintWriter writer;
}
```

Not Using Import

- Import statements can be avoided all together by referring to every class by its full package location
- Can make code harder to read

```
public class Account {  
    javax.swing.JButton b = new javax.swing.JButton("OK");  
}
```

- Occasionally done if a class name is only referenced once

Creating Packages

- All Java classes you create should be in packages
- Package names refer to directory structures
- To create packages, do the following
 1. Create a directory structure to map to your location (usually domain name in reverse)
 2. Place a package declaration at the top of your classes
- Thankfully, IDEs handle all this for you

A Packaged Class

- When compiled, this class must be saved in a directory structure `com\conygre\banking`
- How does the JRE know where this directory structure is?

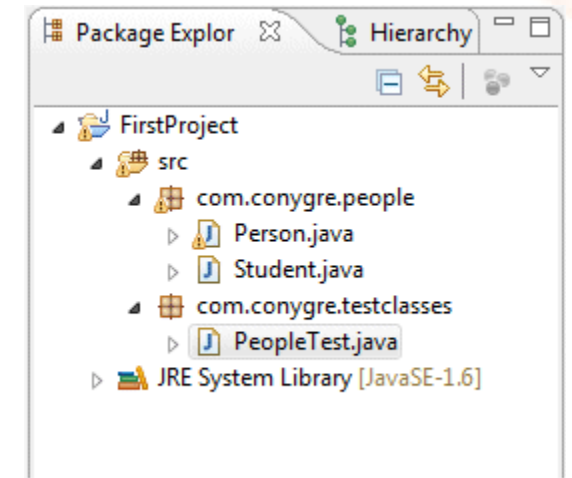
```
package com.conygre.banking;
```

```
import java.awt.*;  
public class Account {  
    ..  
}
```

`com\conygre\banking\Account.java`

IDEs and Packages

- Eclipse shows packaged classes in the **Package Explorer**
- Classes can be moved between packages by simply dragging and dropping
- Both Eclipse and IntelliJ will take care of
 - Creating folders on the hard drive
 - Import statements for moved classes will be changed automatically



Classpath

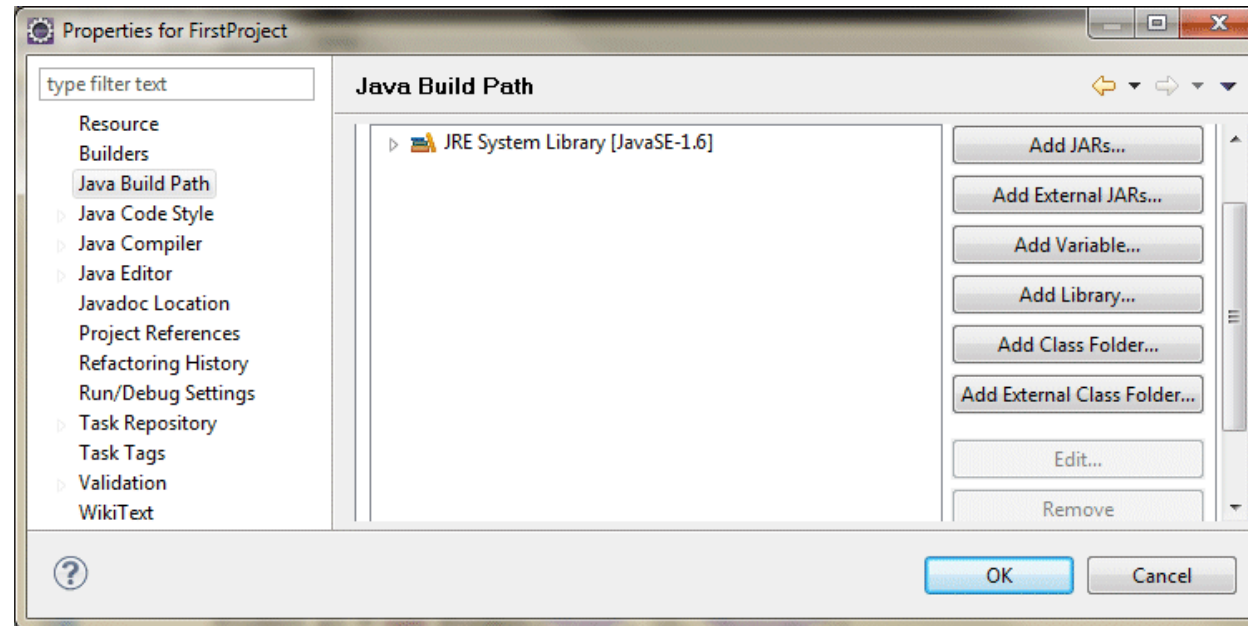
- The JRE uses the `classpath` environment variable to locate classes
- This variable contains directories where packages are found

```
classpath=.;c:\myclasses;c:\yourclasses
```

- The Account class would therefore need to be in a directory structure `c:\myclasses\com\conygre`, or `c:\yourclasses\com\conygre`, or the `current directory\com\conygre`
- The dot in the classpath refers to the current directory

Eclipse and IntelliJ Projects and the Classpath

- Both Eclipse and IntelliJ projects have a **Build Path**, which can be edited to add Jars and class locations to the classpath for the project



- Don't do this for 'proper' projects!!!
 - Use Maven or Gradle

Handling Dependencies

- Adding libraries to Eclipse projects or IntelliJ projects is generally not done in enterprise development environments because
 - It is hard to manage the locations when multiple developers are involved in a project
 - It is very difficult to facilitate automated builds
- Modern applications would use a build tool to facilitate dependency management
 - Gradle
 - Maven

Summary

- Packages are used to organise classes
- The classpath is used to locate classes
- Command Line Options